

软件工程基础第 8 章—软件测试

详细学习笔记

中南大学计算机学院任胜兵

2026 年 5 月 28 日

目录

1	软件测试基础 (Fundamentals of Software Testing)	2
1.1	软件验证 (Software Verification)	2
1.2	软件测试的定义	2
1.3	测试观点 (Testing Perspectives) ——Myers 的观点	3
1.4	测试原则 (Testing Principles)	3
1.5	测试工具 (Testing Tools)	3
1.5.1	静态分析测试软件	3
1.5.2	动态分析测试软件	3
1.5.3	测试数据自动生成程序	4
1.5.4	文件比较程序	4
1.5.5	商业测试工具	4
1.6	测试组织 (Testing Organization)	4
1.7	测试与调试的区别 (Testing vs Debugging)	4
1.8	动态测试步骤 (Dynamic Testing Steps)	4
2	代码复审 (Code Review)	5
2.1	代码复审内容	5
2.2	代码会审 (Code Inspection)	5
2.3	走查 (Walkthrough)	6
2.4	办公桌检查 (Desk Checking)	6
3	白盒测试 (White-Box Testing)	6
3.1	逻辑覆盖法 (Logical Coverage)	6
3.2	基本路径覆盖法 (Basic Path Coverage)	8
3.3	循环覆盖法 (Loop Coverage)	8
3.3.1	简单循环测试策略	8
3.3.2	嵌套循环测试策略	9
3.3.3	串接循环测试策略	9

4 黑盒测试 (Black-Box Testing)	9
4.1 黑盒测试发现的错误类型	9
4.2 测试用例设计方法概述	9
4.3 等价分类法 (Equivalence Partitioning)	9
4.3.1 等价类定义指南	10
4.3.2 等价分类法设计测试用例的步骤	10
4.3.3 示例: 标识符检查程序	10
4.4 边界值分析法 (Boundary Value Analysis)	11
4.4.1 边界值分析指南	11
4.4.2 边界值分析示例	11
4.5 猜错法 (Error Guessing)	11
4.6 因果图法 (Cause-Effect Graphing)	12
4.7 黑盒法综合策略	13
5 单元测试 (Unit Testing)	13
5.1 测试策略	13
5.2 单元测试的作用	13
5.3 测试内容	13
5.3.1 模块接口测试	14
5.3.2 模块局部数据结构测试	14
5.3.3 模块重要执行通路 (路径) 测试	14
5.3.4 模块内部错误处理测试	14
5.3.5 模块边界条件测试	15
5.4 测试的阶段及活动 (GB/T15532-1995)	15
5.5 单元测试的一般流程	15
5.6 单元测试软件 (Drivers and Stubs)	15
6 集成测试 (Integration Testing)	16
6.1 测试内容	16
6.2 测试策略	16
6.3 非渐增式测试 (Non-Incremental Integration)	16
6.4 渐增式测试 (Incremental Integration)	16
6.4.1 自顶向下的渐增式测试 (Top-Down Integration)	17
6.4.2 自底向上的渐增式测试 (Bottom-Up Integration)	17
6.4.3 混合的渐增式测试 (Mixed/Sandwich Integration)	18
6.5 回归测试 (Regression Testing)	18
7 确认测试 (Validation Testing)	18
7.1 确认测试内容	19
7.2 Alpha 测试和 Beta 测试 (Alpha and Beta Testing)	19
7.2.1 Alpha 测试	19
7.2.2 Beta 测试	19

8 系统测试 (System Testing)	20
8.1 恢复测试 (Recovery Testing)	20
8.2 安全性测试 (Security Testing)	20
8.3 可用性测试 (Usability Testing)	21
8.4 安装测试 (Installation Testing)	21
8.5 互连测试 (Interoperability Testing)	21
9 调试 (Debugging)	21
9.1 调试的步骤	21
9.2 调试的难点	22
9.3 调试的策略	22
9.3.1 猜测法 (Guessing)	22
9.3.2 跟踪法 (Backtracking)	22
9.3.3 演绎法 (Deduction)	22
9.3.4 归纳法 (Induction)	22
9.4 调试的原则	23
10 小结 (Summary)	23
附录：核心概念速查表	23

1 软件测试基础 (Fundamentals of Software Testing)

1.1 软件验证 (Software Verification)

软件验证是通过检查和提供客观证据表明软件已经满足规定的需求，是确保软件质量和降低软件成本的重要手段，涉及软件的整个生存周期。[p.2]

进行软件验证的方式大体有两种：**测试 (Testing)** 和 **证明 (Proof)**。[p.2]

测试又分为：

- **静态测试 (Static Testing)**：又称评审 (Review)，是对软件进行的一种分析和检查活动。[p.2]
 - **动态测试 (Dynamic Testing)**：通过运行软件来检验其动态行为和运行结果的正确性。[p.2]
- 证明是一种通过形式化的数学方法来确保软件正确性的活动。[p.2]

1.2 软件测试的定义

软件测试是在软件投入运行前，对软件需求分析、设计规格说明和编码的复审，是为了发现错误，通过检查和提供客观证据表明软件已经满足规定的需求。[p.3]

软件测试是确保软件质量和降低软件成本的重要手段。一般的软件开发组织要将 30%~40% 的项目精力投入到测试之中，一些人命攸关的软件（如航空器的飞行控制软件）其测试费用往往更高。[p.3]

软件错误的来源主要有：[p.4]

- 用户/客户/开发人员之间信息交流不畅；
- 开发人员/管理人员经验与素质缺陷；
- 软件需求不确定（需求模糊或不断变化）；
- 软件及系统复杂（规模大、技术性强）。

测试复杂性与经济性：[p.5]

- 纠正误解：开发一个程序是困难的，测试一个程序则比较容易——这是误解。事实上由于测试情况数量巨大，都不可能进行彻底的测试，即**穷举测试 (Exhaustive Testing)**。
- 为了降低测试成本，选择测试用例时应注意遵守“**经济性**”的原则。
- 测试的主要影响因素：系统的类型、潜在的用户数量、信息的价值、开发机构、测试的时机。

测试存在的疑问：[p.6]

- 测试应该什么时候开始？
- 谁应该执行测试？
- 测试技术如何能够有助于防止错误？
- 测试活动如何被集成到软件过程的模型中？
- 测试如何有利于复用？
- 测试需要多大的工作量？

重要认识：软件测试 $\neq$ 程序测试。[p.7]

- 软件测试对象：软件生存周期各阶段**文档和代码**。
- 据美国一家公司统计：64% 的错误属于分析和设计的错误，编码错误只占 36%。
- 软件测试不是打消软件工程师积极性的过程，而是同软件分析与设计一样，是为了建成高质量的软件，只不过是不同的侧面进行罢了。

1.3 测试观点（Testing Perspectives）——Myers 的观点

G.J. Myers 在其著作《软件测试技巧》中陈述了一系列关于测试的观点：[p.8]

1. 测试是为了寻找错误而运行程序的过程。
2. 一个好的测试用例（Test Case）在于能发现至今未发现的错误。
3. 一个成功的测试是发现了至今未发现的错误的测试。

总结：软件测试就是试图以最少的代价发现软件分析、设计和编码中存在的各种不同类型的错误，从而提高软件质量，降低软件成本。[p.8]

1.4 测试原则（Testing Principles）

软件测试应遵循以下原则：[p.9]

1. 测试应“尽早地和不断地进行”（Test early and continuously）。
2. 较早确定测试计划，严格执行测试计划。
3. 注意错误的群集现象和应用 **Pareto 原则**（80-20 法则：80% 的错误集中在 20% 的模块中）。
4. 测试规模应从小到大。
5. 测试应一般由独立的第三方进行。
6. 应保证测试用例的完整性和有效性。
7. 应保存所有测试用例和出错统计等，直至软件不用为止。

1.5 测试工具（Testing Tools）

软件测试工具是一种测试软件，开发人员借助它可以提高软件测试工作的效率。[p.10]

按工作方式可分为：**静态分析工具**（Static Analysis Tools）和**动态测试工具**（Dynamic Testing Tools）。[p.10]

按功能分有：[p.10]

- **测试计划工具**：支持制订测试计划；
- **测试设计与开发工具**：支持测试数据生成等；
- **测试执行工具**：支持特定的测试方法，支持回归测试等；
- **测试评价工具**：报告测试覆盖情况；
- **测试管理工具**：协助进行测试组织工作；
- **其它辅助测试工具**。

1.5.1 静态分析测试软件

通过扫描被测程序的正文，对其数据流和控制流进行分析，然后送出测试报告。[p.11]

主要功能包括：变量检查、逻辑结构检查、接口检查、编程风格检查和静态特性统计等，通过建立交叉引用表的方式等实现。[p.11]

1.5.2 动态分析测试软件

通过对被测程序有控制地运行，自动地监视、记录和统计被测程序的运行情况。[p.12]

主要功能包括：语句执行次数统计、执行开销估算、CPU 执行时间分析和软硬件资源利用分析等。[p.12]

1.5.3 测试数据自动生成程序

用来为被测程序自动产生测试输入数据。一方面可以减轻生成大量测试数据的负担，另一方面可以避免人为的偏见。对于预期结果，可以利用手工计算等得到。[p.13]

按生成测试数据的方式来分有：路径测试数据生成程序、随机测试数据生成程序、根据数据规格说明生成测试数据等。[p.13]

1.5.4 文件比较程序

用来自动检查测试结果。其检查过程一般主要由三步组成：[p.14]

1. 建立一个存放预期结果的文件；
2. 执行测试，并把测试结果存进另一文件；
3. 使用文件比较程序比较上述两个文件，打印出有差异者的情况。

1.5.5 商业测试工具

Mercury Interactive 公司的产品：[p.15]

- **TestDirector**：基于 Web 的测试管理系统，可以在公司组织内进行全球范围内测试的协调。
- **WinRunner**：企业级的用于检验应用程序是否如期运行的功能性测试工具。
- **LoadRunner**：较高规模适应性的自动负载测试工具，能预测系统行为，优化性能。

其他测试工具如 BoundsChecker、TrueTime、Jtest、C++ Test、CodeWizard、Insure++ 等，分别支持 C++、Java、Visual Basic、C 等语言，功能涵盖错误检查、性能分析、覆盖率统计等。[p.16]

1.6 测试组织 (Testing Organization)

一般来讲，独立测试之前，软件开发者应负责对程序单个模块测试，以保证每个模块能完成详细设计的功能等。[p.17]

在很多情况下，软件开发者也进行集成测试，以保证每个模块能按总体设计的要求形成整个软件系统。[p.17]

在系统形成之后，独立测试小组才开始介入，同时为了保证测试顺利进行，在测试过程中，开发人员必须协助。独立测试应确保系统满足需求分析的要求和用户意图。[p.17]

1.7 测试与调试的区别 (Testing vs Debugging)

测试是查找错误症状的过程，调试则是查找错误症状原因并改正错误的过程。[p.18]

测试与调试的关系图说明：软件配置、测试配置和测试工具共同参与测试过程，输出测试结果、预期结果和错误率数据；调试过程则对出现错误的软件进行修改，产生修改后的软件配置。[p.18]

1.8 动态测试步骤 (Dynamic Testing Steps)

动态测试的步骤与软件开发过程相对应：[p.19]



测试过程按照**单元测试** → **集成测试** → **确认测试** → **系统测试**的顺序由小到大逐步进行。
[p.19]

2 代码复审（Code Review）

代码复审一般在程序通过编译（如查找语法等错误）及静态分析工具检查（如找出程序在结构、编码标准与风格等方面的错误）之后，在动态测试之前进行。[p.20]

代码复审是一种**人工进行的测试**，可以由程序作者本人非正式地进行，也可以由审查小组以开会的方式进行。[p.20]

代码复审的主要方法有：**代码会审**（Code Inspection）、**走查**（Walkthrough）、**办公桌检查**（Desk Checking）。[p.20]

经验表明，代码复审是一种非常有效的程序验证技术，该方法能够有效地发现 30% 到 70% 的逻辑设计和编码错误。[p.20]

2.1 代码复审内容

对源程序代码进行的复审主要着重于检查编码实现是否完备、正确等。[p.21]

在复审过程中，可以对照有关条例或**错误检验表**（Error Checklist），查找程序在结构、功能、编码标准和风格等方面的错误或提出质疑。[p.21]

错误检验表一般包括的内容：**数据引用错误**、**数据声明错误**、**计算错误**、**比较错误**、**控制流程错误**、**接口错误**、**输入/输出错误**等。[p.22-23]

2.2 代码会审（Code Inspection）

会审小组一般由 4 人左右组成，包括组长 1 人，程序作者 1 人，其他程序员（或测试员）1~2 人。会审小组通过对评审材料阅读、讨论和争议，对程序代码进行检查。[p.24]

会审流程：[p.24]

1. **会前：**小组负责人将待评审的材料及有关附件发放给与会者，以熟悉有关内容。
2. **会上：**程序作者逐句朗读并讲解程序代码，其他人则集中精力对照有关条例或检查表，捕捉程序可能存在的问题，也可以展开讨论。
3. **会后：**应把查出的问题清单交给程序作者处理，作者处理完后交回组长。如果问题很多，或者程序改动较大，则在改正后应再次组织会审。

会审一方面可使得同行之间相互启发从而更易于发现错误，另一方面有利于互相学习、交流经验、取长补短，以进一步提高软件开发质量和效率。[p.24]

2.3 走查 (Walkthrough)

走查与代码会审类似，但增加了人工“执行”程序的环节：[p.25]

1. 会前：发放有关材料给与会者进行熟悉，并至少指定一人设计测试用例。
2. 会上：与会者扮演计算机角色，人工“执行”被测程序。通过将测试用例“输入”被测程序，对程序的逻辑和功能提出各种疑问，并进行有关的讨论和争议，以发现程序中存在的问题。
3. 会后的处理同代码会审相同。

2.4 办公桌检查 (Desk Checking)

办公桌检查可以看成是由一个人进行的代码复审。[p.26]

程序作者在程序通过编译之后，进行单元测试之前，对源代码进行分析、检验，并补充有关文档，以发现程序中的错误。[p.26]

其方式可以按照错误检验表来分析被查程序，也可以仿照走查对程序进行人工“执行”。[p.26]

3 白盒测试 (White-Box Testing)

白盒测试是一种以程序的内部逻辑结构为依据设计测试用例的方法，因而又称结构测试 (Structural Testing) 或玻璃盒测试 (Glass-Box Testing)。[p.27]

合理的白盒测试就是要选取足够的测试用例，对源代码实行比较充分的覆盖，以便尽可能多地发现程序中的错误。[p.27]

主要有三种方法：逻辑覆盖法、路径覆盖法和循环覆盖法。[p.27]

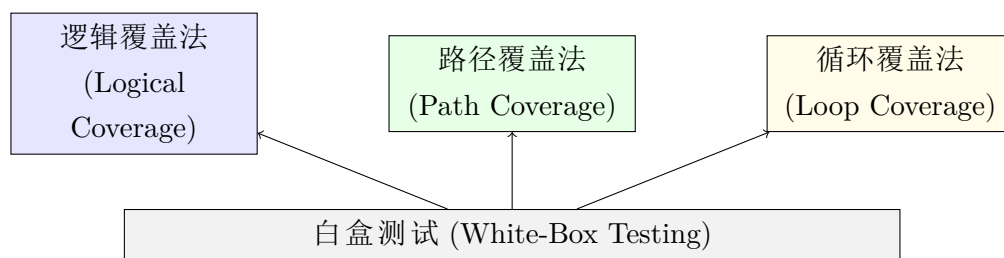


图 1: 白盒测试方法分类

3.1 逻辑覆盖法 (Logical Coverage)

逻辑覆盖法通过对程序逻辑结构的遍历来实现程序的覆盖。从低到高分为以下覆盖级别：[p.28]

- 语句覆盖 (Statement Coverage)：测试用例能使被测程序的每条执行语句至少执行一次。这是最弱的覆盖标准。[p.28]
- 判定覆盖 (Decision Coverage)：测试用例能使被测程序中的每个判定至少取得一次“真”和一次“假”。又称分支覆盖 (Branch Coverage)。[p.28]

- **条件覆盖** (Condition Coverage): 测试用例能使被测程序中每个判定的每个条件至少取得一次“真”和一次“假”。如果判定中只有一个条件, 则条件覆盖便满足判定覆盖, 否则不一定。[p.28]
 - **判定/条件覆盖** (Decision/Condition Coverage): 测试用例既满足判定覆盖, 又满足条件覆盖。[p.28]
 - **条件组合覆盖** (Condition Combination Coverage): 测试用例使每个判定中所有可能的条件取值组合至少执行一次。这是最强的逻辑覆盖标准。[p.28]
- 覆盖程度的强弱关系如下: [p.28]

条件组合覆盖 $\supset$ 判定/条件覆盖 $\supset$ 条件覆盖
 条件组合覆盖 $\supset$ 判定/条件覆盖 $\supset$ 判定覆盖 $\supset$ 语句覆盖

逻辑覆盖法实施步骤: [p.29]

1. 选择逻辑覆盖程度类型;
2. 选择测试路径以满足选定的覆盖程度;
3. 选择测试输入数据以满足选定的测试路径和覆盖程度;
4. 根据测试输入数据和测试路径计算预期结果。

示例: 对于如图所示的程序流程图 (包含判定“ $A > 1$ 且 $B = 0$ ”和“ $A = 2$ 或 $X > 1$ ”), 可以针对不同覆盖标准设计不同的测试用例。[p.30–31]

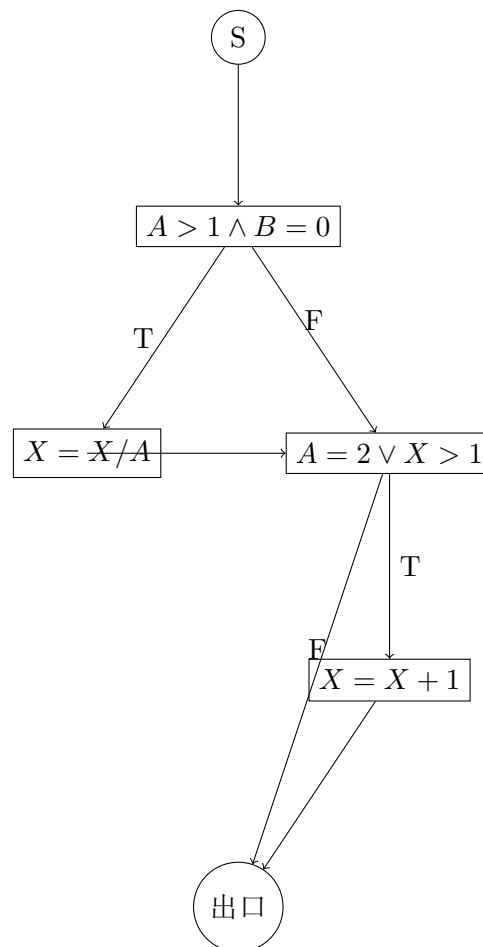


图 2: 白盒测试示例程序流程图 [p.30]

3.2 基本路径覆盖法 (Basic Path Coverage)

逻辑覆盖法并没有检测程序的所有执行路径。基本路径测试是 T. McCabe 首先提出的一种白盒测试技术。所谓**基本路径**是指程序中至少引进一条新的语句或一个新的条件的任一路径。[p.32]

循环处理在计算路径时只计算一次。[p.32]

基本路径测试法是在程序图的基础上,通过分析**环形复杂性** (Cyclomatic Complexity),导出基本路径集,然后设计测试用例使基本路径集中的每条路径至少经过一次。[p.32]

主要步骤: [p.33]

1. 以详细设计结果或源程序代码为基础,导出程序图;(注意应将复合条件判定转化为单一条件判定)。
2. 计算程序图的环形复杂度;(可以采用图形矩阵方法计算,也可采用其它方法来计算)。
3. 确定基本路径集;(基本路径集的路径数就是环形复杂度大小)。
4. 生成测试用例,使基本路径集中的每条路径至少经过一次。

环形复杂度计算公式: [p.35]

$$V(G) = E - N + 2$$

其中 E 为边数, N 为节点数。或者: $V(G) = P + 1$, 其中 P 为判定节点数。

示例: 对于某程序图,环形复杂度为 5,故基本路径数为 5 条。[p.36]

path1: 1-2-4-5-6-7

path2: 1-2-3-4-5-6-7

path3: 1-2-3-8-4-5-6-7

path4: 1-2-3-8-4-5-9-7

path5: 1-2-3-8-4-5-6-9-7

满足基本路径覆盖不一定满足条件组合覆盖,反之亦然。为了实现较充分的覆盖,可以设计测试用例既满足条件组合覆盖,又满足基本路径覆盖。[p.37]

3.3 循环覆盖法 (Loop Coverage)

逻辑覆盖法和基本路径覆盖法对于循环只进行了循环一次的测试,显然是不充分的。而循环是大多数软件实现算法的关键部分,因此对循环的测试是十分重要的。[p.38]

对于结构化程序而言,循环主要有三种:**简单循环** (Simple Loop)、**嵌套循环** (Nested Loop) 和**串接循环** (Concatenated Loop)。[p.38]

3.3.1 简单循环测试策略

对于简单循环,假设 n 是循环的最大次数,则可以采用如下策略设计测试用例: [p.39]

- 整个跳过循环,即循环 0 次;
- 只有一次通过循环,即循环 1 次;
- 循环 2 次;
- 循环 m 次,其中 $m < n$;
- 分别循环 $n - 1$ 、 n 和 $n + 1$ 次。

3.3.2 嵌套循环测试策略

对于嵌套循环，可以采用如下策略来导出测试用例：[p.40]

1. 从最内层循环开始，其他循环设置为最小次数循环；
2. 对最内层循环使用简单循环策略，并为范围外或排除的值增加其他测试；
3. 由内向外构造下一个循环的测试，使其外层循环为最小次数，其内部嵌套循环为“典型”次数，直到所有循环测试完。

3.3.3 串接循环测试策略

对于串接循环：[p.41]

- 如果串接循环的循环都彼此独立，则可以采用简单循环的测试策略；
- 如果循环不独立（如第二个循环变量的初始值是外层第一个循环变量的当前值），则应使用嵌套循环的测试策略。

4 黑盒测试（Black-Box Testing）

黑盒测试又称**功能测试**（Functional Testing）、**数据驱动测试**（Data-Driven Testing）等，它将待测试对象看成是一个黑盒子，不考虑程序内部的逻辑结构和特性，只依据规格说明书检查程序的功能是否能正常使用。[p.42]

通常，白盒测试用于测试的早期，而黑盒测试由于不需了解程序内部情况，因而被许多后期测试（如确认测试、系统测试）采用。[p.42]

4.1 黑盒测试发现的错误类型

黑盒测试常用来发现以下类型的错误：[p.43]

- **功能错误**：表现在功能不正确或功能遗漏或实现了不该实现的功能等。
- **接口错误**：表现在不能正确地接收信息或输出信息。
- **数据错误**：表现在数据结构错误或外部信息（如数据文件）访问错误。
- **性能错误**：表现在性能需求得不到满足。
- **初始化或终止错误**：表现在不能进行正确的初始化或终止。

4.2 测试用例设计方法概述

用黑盒测试发现程序中的错误，主要根据输入条件和输出条件确定测试数据，来检查程序是否能产生正确的输出。[p.44]

进行黑盒测试的主要方法有：等价分类法、边界值分析法、猜错法、因果图法。[p.44]

4.3 等价分类法（Equivalence Partitioning）

等价分类法将所有可能的输入数据（有效的或无效的）划分成若干个**等价类**（Equivalence Class），然后从每个等价类中选出一个作为“代表”形成测试用例。[p.45]

基本假定：等价类中的所有数据对于暴露程序中的错误是等效的。[p.45]

在确定输入数据的等价类时常要分析输出数据的等价类，以便根据输出数据的等价类导出对应的输入数据等价类。[p.45]

4.3.1 等价类定义指南

下述等价类定义指南有助于等价类的划分：[p.46]

1. 如果输入条件规定了一个取值范围或值的个数，则可以定义一个有效等价类和两个无效等价类。
2. 如果输入条件是一个布尔量，则可以确定一个有效等价类和一个无效等价类。
3. 如果规定了输入值的集合，或者规定了“必须如何”的条件，则可以定义一个有效等价类和一个无效等价类。
4. 如果规定了输入数据必须遵守的规则，则可以定义一个有效等价类（符合规则）和若干无效等价类（从不同角度违反规则）。
5. 如果规定了输入数据的一组值，而且程序对不同输入值做不同处理，则可以定义若干有效等价类（每个值一个有效等价类）和一个无效等价类。
6. 如果确知已划分的等价类中各元素在程序中的处理方式是不同的，则应将此等价类进一步划分成更小的等价类。

4.3.2 等价分类法设计测试用例的步骤

1. 划分等价类，形成等价类表：[p.47]
2. 为每个等价类规定一个唯一的编号；
3. 设计一个新的测试用例，使其尽量多地覆盖尚未被覆盖的有效等价类，重复这一步，直到所有的有效等价类都被覆盖为止。
4. 设计一个新的测试用例，使其覆盖一个而且只覆盖一个无效等价类，重复这一步，直到所有无效等价类均被覆盖为止。

4.3.3 示例：标识符检查程序

规格说明：在某一 C 语言版本中规定：“标识符是以字母或下划线开头，后跟字母、数字或下划线的任意组合构成；有效字符数为 16 个；且标识符不能是保留字。”[p.48]

等价类划分：[p.49]

表 1: 等价类划分表 [p.49]

输入条件	有效等价类	无效等价类
标识符首字符	(1) 以字母开头 (2) 以下划线开头	(3) 以非字符非下划线开头
标识符组成	(5) 由字母、数字或下划线构成	(7) 由非字母、非数字或非下划线构成
是否保留字	(4) 不是保留字	(6) 是保留字
字符个数	(8) 为 1 到 16 个	(9) 字符个数为 0 个 (10) 字符个数大于 16 个

有效等价类测试用例（尽量公用）：[p.50]

- 输入 X1: 包含有效等价类 (1)、(4)、(5)、(8)，预期结果：有效

- 输入 `_2`: 包含有效等价类 (2)、(4)、(5)、(8), 预期结果: 有效
 无效等价类测试用例 (每类一个): [p.51]
- 输入 `$X1`: 包含无效等价类 (3), 预期结果: 无效, 以非字母非下划线打头
- 输入 `FOR`: 包含无效等价类 (6), 预期结果: 无效, `FOR` 是保留字
- 输入 `X$R`: 包含无效等价类 (7), 预期结果: 无效, 由非字母非数字且非下划线构成
- 输入 `(无)`: 包含无效等价类 (9), 预期结果: 无效, 标识符不能为空
- 输入 `X123456789ABCDEFGH`: 包含无效等价类 (10), 预期结果: 无效, 标识符字符个数大于 16 个

4.4 边界值分析法 (Boundary Value Analysis)

经验表明, 程序在处理边界情况时最容易发生错误, 从而设计测试用例使程序运行在边界情况附近的测试方案, 暴露出程序错误的可能性会更大些。[p.52]

边界值分析法主要用来选择等价类边界值作为测试用例检查程序边界运行情况, 是一种补充等价分类法的测试用例设计技术。[p.52]

通常, 输入等价类和输出等价类的边界情况是测试的重点, 应当选择**正好等于、刚刚小于和刚刚大于**边界的值作为测试数据。[p.52]

4.4.1 边界值分析指南

1. 如果输入条件代表以 a 和 b 为边界的范围, 则测试用例应当包含 a 、 b 、略小于下界 a 、略大于上界 b 的值。[p.53]
2. 如果输入条件规定了值的个数, 则用最大个数、最小个数、比最大个数多 1、比最小个数少 1 的数作为测试数据。[p.53]
3. 上述两条指南对输出条件也适用。[p.53]
4. 如果程序的规格说明中给出的输入和输出域是有序集合 (如有序表), 则应选取集合的第一个元素和最后一个元素作为测试数据。[p.53]
5. 如果程序数据结构有预定义的边界 (如数组有 100 项, 下标下界是 0, 上界是 99), 则应选择测试数据测试其边界的数据项。[p.53]

4.4.2 边界值分析示例

以标识符检查程序为例, 设计以下边界值测试用例: [p.54]

- 输入 `X` (字符数为 1, 且以字母开头): 预期结果: 有效
- 输入 `_123456789ABCDEF` (以下划线开头, 字符数为 16): 预期结果: 有效
- 输入 `_1` (以下划线开头, 字符数为 1): 预期结果: 有效
- 输入 `X123456789ABCDEF` (以字母开头, 字符数刚好等于 16 个): 预期结果: 有效
- 输入 `_123456789ABCDEF1` (以下划线开头, 字符数刚好大于 16 个): 预期结果: 无效
- 输入 `X123456789ABCDEF1` (以字母开头, 字符数刚好大于 16 个): 预期结果: 无效

4.5 猜错法 (Error Guessing)

等价分类法和边界值分析法通过选择有代表性的测试数据来暴露程序错误。但不同类型不同特点的程序通常又有一些特殊的容易出错的情况。并且, 有时分别使用某些测试用例测试时

程序工作正常，但其组合可能会使程序出错。[p.55]

测试人员应依靠经验和直觉，从各种可能的方案中选出最可能引起程序出错的方案。[p.55]

猜错法能充分发挥人的经验和直觉，在一个测试小组中集思广益，方便实用，特别是在软件测试基础较差的情况下，很好地组织测试小组（也可包括外来人员）进行错误猜测，是一种很有效的测试方法。[p.55]

示例：测试一个排序程序，可先用边界值分析法设计测试用例，再用猜错法补充。例如：[p.56]

- 边界值法：(1) 排序序列为空；(2) 排序序列仅有一个数据；(3) 排序序列为满序列数据。
- 猜错法补充：(4) 排序序列已经按要求排好序；(5) 排序序列的顺序与要求的顺序恰好相反；(6) 排序序列中的所有数据全部相等；等等。

4.6 因果图法 (Cause-Effect Graphing)

猜错法是根据经验和直觉设计测试用例，尽管能考虑到输入组合的情况，但显得不充分。[p.57]

因果图法借助图形来设计测试用例，特别适用于被测程序具有多种输入条件，程序的输出又依赖于输入条件的各种组合的情况。[p.57]

因果图中使用的基本符号包括：[p.58]

- **原因 (Cause)：**用节点 I 表示输入条件；
- **结果 (Effect)：**用节点 O 表示输出条件；
- **恒等 (Identity)：**若原因成立，则结果成立；
- **非 (NOT)：**若原因成立，则结果不成立；
- **或 (OR)：**若多个原因中至少一个成立，则结果成立；
- **与 (AND)：**若多个原因都成立，则结果成立。

因果图法步骤：[p.59]

1. 分析程序规格说明中哪些是原因，哪些是结果。原因常常是输入条件或输入条件的等价类，结果则是输出条件。
2. 分析程序规格说明中描述内容的语义和限制，找出两类关系，画出因果图。
3. 把因果图转换成判定表 (Decision Table)。
4. 对判定表的每一列写成一个测试用例。

示例：假设某程序规格规定：“第一个字符必须是字母 I 或 O ，第二个字符必须是一个数字，在此情况下对文件进行处理。如果第一个字符不正确，则给出操作类型错信息。若第一个正确，但第二个字符不正确，则给出顺序号错信息。” [p.60]

原因和结果分析：[p.61]

- 原因 I_1 ：第一个字符是 I ；
- 原因 I_2 ：第一个字符是 O ；
- 原因 I_3 ：第二个字符是数字；
- 结果 O_1 ：对文件进行处理；
- 结果 O_2 ：给出“操作类型错”信息；
- 结果 O_3 ：给出“顺序号错”信息。

因果图表达： $O_1 = (I_1 \vee I_2) \wedge I_3$ ； $O_2 = \neg(I_1 \vee I_2)$ ； $O_3 = (I_1 \vee I_2) \wedge \neg I_3$ 。[p.62]

转换为判定表后，每一列对应一个测试用例：[p.63-64]

- I_4 ：对文件进行处理

- O5: 对文件进行处理
- XX: 操作类型错
- X8: 操作类型错
- IX: 顺序号错
- OX: 顺序号错

4.7 黑盒法综合策略

综合运用黑盒测试方法时，建议按以下顺序：[p.65]

1. 首先用边界值分析法设计测试用例；
2. 必要时用等价分类法补充测试用例；
3. 必要时再用猜错法补充测试用例；
4. 如果在程序的说明中含有输入条件的组合，宜在一开始就采用因果法，然后再按上述步骤进行。

5 单元测试（Unit Testing）

单元测试又称模块测试（Module Testing）或分调，是动态测试中的第一步，通常在编码阶段进行。[p.66]

单元测试集中检查软件设计的最小单元——模块，即程序中最小的独立编译单位。在源程序代码经过编译、评审，确认没有语法错误之后，便可开始进行单元测试的测试用例设计，以发现程序内部逻辑结构的错误。[p.66]

单元测试可以由程序作者进行，也可以由同行互测对方的单元进行。[p.66]

5.1 测试策略

单元测试一般总是把白盒法和黑盒法结合运用。[p.67]

- 先用黑盒法设计出一组基本的测试用例；
- 然后用白盒法，根据覆盖标准要求补充新的测试用例满足覆盖标准。

在一般情况下，单元测试应以白盒法为主。[p.67]

5.2 单元测试的作用

采用从单元开始而不是对整个程序进行测试，一方面可以减少测试工作的复杂性，另一方面容易进行错误定位和纠错。同时，对多个模块的测试可以并行地进行，从而缩短测试的周期。[p.68]

性价比好，因为错误发现越早，修复成本越低。[p.68]

5.3 测试内容

单元测试在于考察模块的接口和内部结构，检查是否符合程序规格说明（即详细设计说明书）的要求。测试的重点在以下几个方面：[p.69]

1. 模块的接口（Module Interface）；
2. 局部数据结构（Local Data Structure）；

3. 重要的执行通路 (Important Execution Paths);
4. 出错处理路径 (Error Handling Paths);
5. 影响以上各项的边界条件 (Boundary Conditions)。

5.3.1 模块接口测试

检查进出模块的数据是否正确。检查清单包括: [p.70]

- 模块的实际输入与定义的输入是否一致 (个数、类型、顺序);
- 全局变量的定义和用法在各个模块中是否一致;
- 使用其他模块时, 是否检查可用性和处理结果;
- 使用外部资源时, 是否检查可用性并及时释放资源 (内存、文件、硬盘、端口等)。

5.3.2 模块局部数据结构测试

检查局部数据结构能否保持完整性。检查清单包括: [p.71]

- 变量从来没有被使用;
- 可能别的地方使用了错误的变量名;
- 不正确或者不一致的说明;
- 变量没有初始化;
- 错误的类型转换;
- 数组越界;
- 非法指针;
- 变量或函数名称拼写错误;
- 使用了外部变量或函数。

5.3.3 模块重要执行通路 (路径) 测试

检查由于计算错误、判定错误、控制流错误导致的程序错误。检查清单包括: [p.72]

- 死代码 (Dead Code);
- 错误的计算优先级;
- 精度错误;
- 比较运算错误 (>、>=、=、==、!=);
- 赋值错误;
- 表达式的不正确符号;
- 循环变量的使用错误;
- 错误赋值。

5.3.4 模块内部错误处理测试

检查内部错误处理设施是否有效。检查清单包括: [p.73]

- 是否检查错误出现 (资源使用前后、其他模块使用前后);
- 出现错误, 是否进行错误处理 (抛出错误、通知用户、进行记录);
- 错误处理是否有效 (在系统干预前处理、报告和记录的错误真实详细)。

5.3.5 模块边界条件测试

检查临界数据是否正确处理。检查清单包括：[p.74]

- 普通合法数据是否正确处理；
- 普通非法数据是否正确处理；
- 边界内最接近边界的（合法）数据是否正确处理；
- 边界外最接近边界的（非法）数据是否正确处理。

5.4 测试的阶段及活动（GB/T15532-1995）

按照国家标准 GB/T15532-1995，单元测试分为三个阶段：[p.75]

1. 完善测试计划阶段：制定方法、资源及进度的计划；确定需测试的与需求有关的特性；细化计划。
2. 获得测试用例集阶段：设计测试用例集；执行测试计划及实现设计。
3. 评价测试单元阶段：执行测试规程；核对终止情况；评价测试效果和被测单元。

5.5 单元测试的一般流程

单元测试的典型流程为：[p.76]

测试计划 → 测试设计 → 测试执行 → 测试记录 → 分析缺陷跟踪 → 测试总结

如果分析测试记录时发现与预期结果不同，则确定并重现缺陷，然后重新回到测试设计环节。[p.76]

5.6 单元测试软件（Drivers and Stubs）

由于一个模块本身不是一个独立的程序，在单元测试时需要为它开发驱动模块（Driver）和桩模块（Stub）。[p.78]

- 驱动模块（Driver Module）：应完成接收测试数据，并把数据传给被测模块，然后打印有关结果等任务。
- 桩模块（Stub Module）：应该模拟实际模块完成少量数据处理，并检验和打印入口处的信息，然后将控制返回给被测模块。

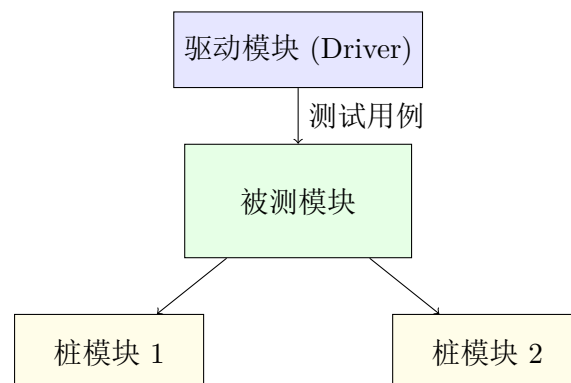


图 3: 单元测试的测试环境 [p.78]

6 集成测试 (Integration Testing)

集成测试, 又称**组装测试**、**综合测试**或**联调**, 是在单元测试完成之后, 将所有模块按概要设计要求组装成系统的时候进行的测试。[p.79]

主要目标是发现与**接口**有关的问题。[p.79]

集成测试有**组装**和**检验**两重意义: 一方面将各经过单元测试的模块拼装起来形成完整可运行的系统; 另一方面要检验每一步拼装过程是否正确。[p.79]

6.1 测试内容

主要检查模块接口和全局数据。检查清单: [p.80]

- 穿越模块接口的数据是否会丢失;
- 一个模块是否对另一个模块产生不利的副作用;
- 模块组装后, 能否达到预期要求的功能;
- 全局数据结构在引用中是否有问题;
- 模块的误差积累是否可以接受。

6.2 测试策略

集成测试一般应由**独立的测试小组**进行, 由测试小组提出测试用例, 记录测试结果, 并编制测试报告。[p.81]

测试用例的设计通常采用**黑盒法**, 其实施策略又分为**非渐增式** (Non-Incremental) 和**渐增式** (Incremental) 两种。[p.81]

回归测试 (Regression Testing): 在模块组装起来后, 新的数据流路径建立起来了, 新的控制逻辑可能激活了等等。这些改变可能会使原本工作正常的功能产生错误, 因此, 应对某些已经进行过的测试的测试用例再重新执行一遍, 以保证上述改变不会传播意外的副作用。[p.81]

6.3 非渐增式测试 (Non-Incremental Integration)

采用非渐增式测试, 一般应先经过单元测试, 然后再把所有模块一次性组装在一起进行测试, 最终得到要求的软件系统。[p.82]

缺点: [p.82]

- 将模块一次性组装在一起运行成功的可能性并不大。其结果往往是发现有错误, 但由于程序中模块一次性引入过多, 难于进行错误定位。
- 同时, 一旦修正错误之后, 新的错误很可能马上会出现。

除规模很小的程序, 一般很少采用此种测试策略。[p.82]

6.4 渐增式测试 (Incremental Integration)

渐增式测试采用逐步加入模块或功能簇的方式进行, 在加入过程中边连接边测试, 比较容易定位和修正错误, 且接口也可以更容易进行彻底地测试。[p.83]

按照添加模块的方式, 又可分为**自顶向下的渐增测试**和**自底向上的渐增测试**。[p.83]

6.4.1 自顶向下的渐增式测试 (Top-Down Integration)

首先集成主控模块，然后按照软件结构的控制层次自上而下进行集成，把主控模块的直接（或间接）调用模块按**深度优先**（Depth-First）或**广度优先**（Breadth-First）的方式集成到整个软件结构中。[p.84]

步骤： [p.85]

1. 以主控模块为被测模块兼驱动模块，所有被主控模块直接调用的下层模块全部用桩模块代替，对主控模块进行测试。
2. 根据集成的实现方法（如深度或广度优先），用实际模块替换相应桩模块，再用桩模块代替它们的直接下属模块。
3. 在每一个模块集成的时候都要进行测试。
4. 进行回归测试（即重新执行以前做过的全部测试或部分测试），排除组装过程中引入新的错误。
5. 重复 (2)~(4)，直到所有的模块都已组装到系统中。

示例： 对于一个模块层次结构（M1 为顶层，调用 M2、M3、M4；M2 调用 M5、M6；M4 调用 M7、M8；M5 调用 M9）：[p.86]

- 深度优先组装顺序：M1 → M2 → M5 → M9 → M6 → M3 → M4 → M7 → M8
- 广度优先组装顺序：M1 → M2 → M3 → M4 → M5 → M6 → M7 → M8 → M9

特点： [p.87]

1. 能较早地显示整个程序的轮廓，对增强开发人员的信心，取得用户的支持，有重要的作用；
2. 只需编写桩模块供测试用，驱动模块可以利用实际模块；
3. 能尽早发现主要控制中的问题，减少以后的返工；
4. 可先对逻辑输入的分支进行组装和测试，检查和克服潜藏的错误，为其后对主加工分支的组装和测试提供了保证；
5. 由于每添加一个新模块，就进行一次测试，这样虽然耗费一些时间，但却使被测程序测试更彻底，尤其是上层模块。

存在逻辑次序问题： 最常见的是出现在当高层测试需要首先对较低层次进行足够测试后才能完成的时候。为了能够准确地实施测试，应当让桩模块正确而有效地模拟下层被调模块的功能和合理的接口，不能是只包含返回语句或只显示该模块已调用信息、不执行任何功能的哑模块。[p.88]

解决办法： 把测试推迟到桩模块用实际模块替代之后进行，或者采用自底向上组装和测试软件。[p.88]

6.4.2 自底向上的渐增式测试 (Bottom-Up Integration)

步骤： [p.89]

1. 把低层模块组合成实现某个特定的子功能的模块簇（Cluster），并用编写的驱动模块控制它进行测试；可以对若干功能簇并行进行测试；
2. 用实际模块换掉驱动模块，沿软件结构自下而上移动，把子功能簇组合起来形成更大的子功能簇，并进行测试；
3. 重复 (2) 直到所有模块组装完毕。

特点： [p.91]

1. 不能在测试的早期显示出程序的轮廓。程序的总体结构，要等到加入最后一个模块时才能最终形成；
2. 测试软件只需要驱动模块，不需要桩模块，而且随着组装层次的上移，驱动模块将大为减少（在多数情况下，编写驱动模块要比桩模块容易）；
3. 由于从低层模块开始组合，所以较易产生测试用例。

6.4.3 混合的递增式测试（Mixed/Sandwich Integration）

混合策略结合了自顶向下和自底向上两种方法的优点：[p.92]

1. 对上层模块采取自顶向下测试，使之能较早期地显示系统的总体轮廓；
2. 对某些关键模块或子系统采用由底向上组装和测试的方法，以便一方面能减少对模块的重复测试次数，另一方面能较容易产生测试用例。

所谓**关键模块**是指：(1) 具有输入/输出功能的模块；(2) 有重要功能、性能或含有新算法的模块。[p.92]

6.5 回归测试（Regression Testing）

采用软件改动前测试时执行过的测试用例对改动后的软件再进行测试。[p.93]

回归测试的测试用例有三种不同类型：[p.93]

1. 能够测试软件的所有功能的**代表性测试用例**；
2. 针对可能会被修改所影响的软件功能而进行附加测试的测试用例；
3. 针对修改过的软件成分进行测试的测试用例。

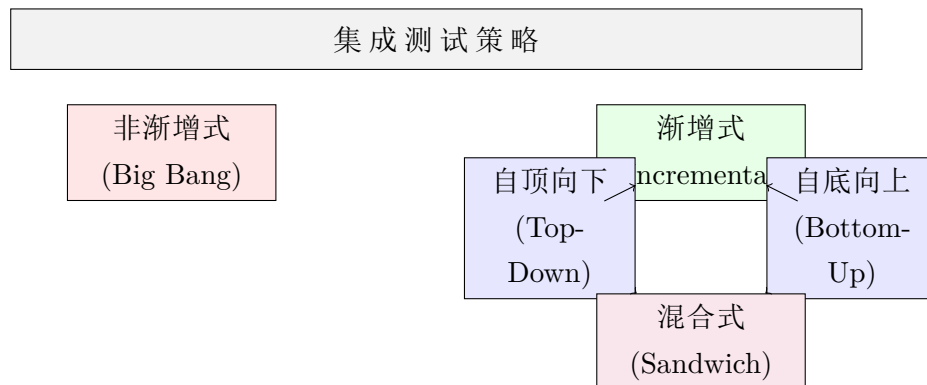


图 4: 集成测试策略分类

7 确认测试（Validation Testing）

所谓确认测试就是验证所开发软件的功能和性能及其他特性是否符合软件需求规格说明书的要求。所以，确认测试又称之为**有效性测试**（Validation Testing）。[p.94]

一般在模拟的环境（也可能就是开发的环境）下，运用**黑盒法**进行测试。[p.94]

确认测试是由软件开发单位组织进行的最后一次测试，也是把软件交给用户，进行正式的安装和验收之前所作的一次重要的准备。为了确保测试质量，一方面应组织独立的测试小组进行测试，另一方面吸收任务委托单位及用户代表参加测试，以提高测试的可信度。[p.94]

7.1 确认测试内容

确认测试的主要内容包括：[p.95]

- **功能测试**（Functional Testing）：验证软件的功能是否符合需求；
- **性能测试**（Performance Testing）：验证软件的性能指标是否达标；
- **强度测试**（Stress Testing）：验证软件在极限条件下的表现；
- **配置复审**（Configuration Review）：检查软件配置是否齐全、正确。

7.2 Alpha 测试和 Beta 测试（Alpha and Beta Testing）

软件开发者要预见用户是如何实际地使用软件实质上是不可能的。例如，命令的使用可能被误解；出现异常的数据组合；输出对测试者或开发者来说似乎是很清晰的，但对这个领域里的用户来说可能是无法理解的；等等。[p.96]

Alpha 测试和 Beta 测试的测试方法，用以发现可能只有最终用户才能发现的错误。[p.96]

7.2.1 Alpha 测试

Alpha 测试是由一个用户在开发环境下进行的测试；也可以是开发机构内部的用户在模拟实际操作环境下进行的测试，软件在一个自然设置状态下使用。软件在开发者对用户的“指导”下进行测试，开发者负责记录错误和使用中出现的问题。显然，Alpha 测试是在一个受控的环境中进行。[p.97]

Alpha 测试的目的是评价软件产品的功能（Functionality, F）、局域化（Localization, L）、可使用性（Usability, U）、可靠性（Reliability, R）、性能（Performance, P）和支持（Support, S）等方面的特性，尤其注重产品的界面和特色。[p.97]

Alpha 测试可以从软件编码结束之时开始，或在模块或集成测试完成之后开始，也可以在确认测试过程中产品达到一定的稳定和可靠程度之后再开始。[p.97]

7.2.2 Beta 测试

Beta 测试是由软件的最终用户（多个）在一个或多个用户场所来进行。这些用户是与软件厂商签定了支持产品预发行合同的外部客户，要求用户使用该产品，并愿意返回有关问题给开发者。一般地，Beta 测试时开发者通常不在测试现场，软件是在开发者不能控制的现场中应用。[p.98]

在 Beta 测试中，由用户负责记下遇到的所有问题，包括主观认定的和真实的问题，定期向开发者报告，开发者在综合用户的报告之后进行修改，最后将软件产品交付给全体用户使用。[p.98]

重要注意事项：[p.98]

- 只有当 Alpha 测试达到一定的可靠程度时，才能开始 Beta 测试。
- Beta 测试的主要目标是测试**可支持性**，包括文档、客户培训等。
- 一般 Beta 测试应尽可能由主持产品发行的人员来管理。

表 2: Alpha 测试与 Beta 测试对比

比较维度	Alpha 测试	Beta 测试
测试地点	开发环境（受控）	用户现场（不受控）
测试人员	开发机构内部用户	最终用户（外部客户）
开发者参与	开发者在场指导	开发者通常不在场
环境控制	受控环境	不受控环境
测试目标	评价 F/L/U/R/P/S 特性	测试可支持性（文档、培训）
开始时机	编码结束/模块测试完成后	Alpha 测试达到可靠程度后

8 系统测试（System Testing）

系统测试是在更大范围内进行的测试，它将经过确认测试的软件作为整个基于计算机的系统的一个元素，与计算机硬件、外设、支持软件、数据和人员等其他系统元素结合在一起，在实际运行环境下，对系统进行的一系列集成和确认测试。[p.99]

系统测试通常由任务委托单位或用户组织的验收小组负责，一般应根据需求分析说明书来设计测试用例，在实际使用环境中运行。[p.99]

系统测试的内容对不同的系统各不相同，但通常包括以下几种类型。[p.99]

8.1 恢复测试（Recovery Testing）

恢复测试就是要通过各种手段，让软件强制性地发生故障，然后验证恢复是否能正常进行的一种系统测试方法。[p.100]

- 如果系统的恢复是**自动的**（由系统本身来进行），则应对重新初始化、数据恢复、重新启动等逐个进行正确性评价。[p.100]
- 如果恢复需要**人工干预**，则需对修复的平均时间进行评估以判定它是否在允许的范围之内。[p.100]

8.2 安全性测试（Security Testing）

安全测试用来验证集成在系统内的保护机制是否能够在实际中保护系统不受到非法侵入。[p.101]

主要破坏方法（测试人员模拟攻击）：[p.101]

1. 攻击易受损坏的部分，破坏安全性；
2. 有目的地引发系统出错，期望在系统恢复过程中侵入系统；
3. 通过各种手段，获取系统的密码；
4. 浏览非保密数据，从中找到进入系统的方法；等等。

若有足够的时间和资源，好的安全性测试应能最终侵入系统。[p.101]

系统设计者的任务是：把系统设计为攻破系统的代价远大于攻破系统后从中得到的好处。[p.101]

8.3 可用性测试 (Usability Testing)

可用性测试主要从使用的方便性、易理解性和易学性等方面对系统进行检查，以发现人为因素或使用习惯等问题。例如：[p.102]

- 用户界面便于使用；
- 出错信息能帮助用户解决问题；
- 销往国外的产品有译本；
- 输入可容错；
- 输出信息有意义并且前后一致；
- 等等。

8.4 安装测试 (Installation Testing)

安装测试是要找出系统安装过程中出现的问题。例如：[p.103]

- 硬件的配置是否合理；
- 用户选择的一套任选方案是否相容；
- 系统是否产生了所需的文件；
- 系统是否每一部分都齐全；
- 等等。

8.5 互连测试 (Interoperability Testing)

互连测试是要验证两个或多个不同的系统之间的互操作性。这类测试对支持标准规格说明，或承诺支持与其他系统互连的软件系统有效。[p.104]

9 调试 (Debugging)

调试，又称**纠错**或**排错**，是程序测试后开始的工作，主要任务是依据测试发现的错误迹象确定错误位置和原因，并加以改正。[p.105]

调试活动由两部分组成：[p.105]

1. 确定程序中可疑错误的确切性质和位置；
2. 对程序（设计，编码）进行修改，排除这个错误。

调试是通过现象，找出原因的一个思维分析的过程。[p.105]

9.1 调试的步骤

调试的一般步骤为：[p.106]

1. 从错误的外部表现形式入手，确定程序中出错位置；
2. 研究有关部分的程序，找出错误的内在原因；
3. 修改设计和代码，以排除这个错误；
4. 重复进行暴露了这个错误的原始测试或某些有关测试，以确认该错误是否被排除；是否引进了新的错误。
5. 如果所做的修正无效，则撤消这次改动，重复上述过程，直到找到一个有效的解决办法为止。

9.2 调试的难点

调试过程中的主要难点包括：[p.107]

- 错误的症状和引起错误的原因可能相隔很远，尤其是在高度耦合的程序结构中；
- 错误症状可能在另一错误被纠正后消失或暂时性的消失；
- 错误症状可能实际并不是由错误引起的（如舍入误差）；
- 错误症状可能是由不易跟踪的人工操作引起的；
- 错误症状可能是和时间相关的，而不是处理问题；
- 很难再现产生错误症状的输入条件；
- 错误症状可能时有时无（如在软硬件结合的嵌入式系统中常常遇到）；
- 错误症状可能是由于把任务分布在若干不同处理器上运行而造成。

9.3 调试的策略

9.3.1 猜测法（Guessing）

该方法通过分析错误症状，根据以往经验，辅助使用已有的计算机工具，猜测错误的原因并进行定位。可以通过以下方式实现：[p.108]

- 在程序中插入打印语句；
- 使用注释或 GOTO 语句运行部分程序；
- 使用调试工具（Debugger）。

9.3.2 跟踪法（Backtracking）

先分析错误征兆，确定最先发现“症状”的位置。然后，人工沿程序的控制流程，向回追踪源程序代码，直到找到错误根源或确定错误产生的范围。[p.109]

跟踪法对于小程序很有效，往往能把错误范围缩小到程序中的一小段代码；仔细分析这段代码不难确定出错的准确位置。但对于大程序，由于回溯的路径数目较多，回溯会变得很困难。[p.109]

9.3.3 演绎法（Deduction）

演绎法排错是测试人员首先根据已有的测试用例，设想及枚举出所有可能出错的原因作为假设；然后再用原始测试数据或新的测试，从中逐个排除不可能正确的假设；最后，再用测试数据验证余下的假设确是出错的原因。[p.110]

演绎法流程：[p.110]

列举可能的原因 → 排除不会发生的原因 → 细析余下的原因 → 证明假设 → 纠正错误

如果无剩余原因，则收集更多的测试结果。如果不能证明假设，则回到收集更多测试数据的步骤。[p.110]

9.3.4 归纳法（Induction）

归纳法排错的基本思想是：从一些线索（错误征兆）着手，通过分析它们之间的关系来找出错误。[p.111]

归纳法流程：[p.111]

收集有关数据 → 组织数据 → 研究数据间的关系 → 提出假设 → 证明假设 → 纠正错误

如果不能证明假设，则回到研究数据间关系的步骤。[p.111]

9.4 调试的原则

- 综合起来，在进行调试时，调试人员应该做到：[p.112]
- 1. **冷静多思：**调试是软件开发过程中最艰巨的脑力劳动之一。错误症状同错误原因之间往往没有明显的联系。这就要求调试人员要有耐心，并且细心，在面临困惑的情况下虚心向同行请教。
 - 2. **彻底修改：**一方面应注意其周围是否存在类似的其它错误，即考虑错误的群集现象；另一方面要纠正错误本身而不是其症状。
 - 3. **防止引入新的错误：**进行回归测试是一种较好的办法。

10 小结（Summary）

软件测试的根本任务就是发现软件中存在的错误。[p.113]

由于软件错误往往出现在软件的分析 and 设计中，而且错误发现越晚，改正错误的工作量也越大，因而在开发前期利用静态测试非常重要，而且发现错误的效率也往往比较高。[p.113]

针对测试中发现的错误进行改正，这便是调试的工作。测试和调试往往交替进行。[p.113]

重要结论：测试只能说明软件中存在错误，不能表明程序没有错误，因而任何软件经过测试后不能保证软件中不再存在错误。[p.113]

附录：核心概念速查表

表 3: 测试方法分类汇总		
分类维度	类别	主要方法/内容
按是否运行程序	静态测试	代码复审（会审、走查、办公桌检查）
	动态测试	白盒测试、黑盒测试
按测试阶段	单元测试	以白盒法为主，测试模块内部逻辑
	集成测试	渐增式（自顶向下/自底向上/混合）/非渐增式
	确认测试 系统测试	黑盒法，Alpha 测试、Beta 测试 恢复、安全、可用性、安装、互连测试
按是否了解内部结构	白盒测试	逻辑覆盖、基本路径覆盖、循环覆盖
	黑盒测试	等价分类、边界值分析、猜错法、因果图法

表 4: 白盒测试覆盖标准对比

覆盖标准	强度	说明
语句覆盖	最弱	每条语句至少执行一次
判定覆盖	较弱	每个判定的真/假分支至少各执行一次
条件覆盖	中等	每个条件的真/假取值至少各执行一次
判定/条件覆盖	较强	同时满足判定覆盖和条件覆盖
条件组合覆盖	最强	每个判定的所有条件取值组合至少执行一次
基本路径覆盖	较强	每个基本路径至少执行一次（基于环形复杂度）

表 5: 黑盒测试方法对比

方法	核心思想	优点	适用场景
等价分类法	划分等价类，选代表值	系统化、减少用例数	输入数据有明确分类
边界值分析法	测试边界附近的值	高效发现边界错误	补充等价分类法
猜错法	凭经验猜测易错位置	发挥人的直觉	经验丰富的测试团队
因果图法	分析输入条件组合	系统分析输入组合	输入条件有组合依赖